Propuesta de Ingeniería: Private Cloud & IoT Smart Display

Documento de Viabilidad, Arquitectura de Sistemas y Stack Tecnológico

Área: DevOps / SysAdmin / IoT **Hardware:** Portátil x86-64 + Galaxy Tab 4 **Objetivo:** Self-Hosting, Privacidad y Portafolio Profesional

1. Resumen Ejecutivo y Objetivos de Ingeniería

El proyecto consiste en la transformación y puesta en producción de dos dispositivos catalogados en obsolescencia tecnológica: un **ordenador portátil x86-64** reconvertido en nodo de servidor central doméstico y una **tablet Samsung Galaxy Tab 4** adaptada como marco digital IoT interactivo. Se elimina la dependencia de servicios de suscripción (Google Photos, Netflix/Plex Pass), garantizando soberanía de datos y ofreciendo una solución reproducible con contenedores Docker, redes seguras y microservicios a medida.

2. Análisis de Viabilidad de Hardware

Componente	Especificación Base	Evaluación Técnica y Viabilidad	Estrategia de Mitigación
Servidor (Portátil)	CPU Dual/Quad Core x86_64, 4-8 GB RAM	**ALTA VIABILIDAD** PostgreSQL, Redis, Immich Core y Jellyfin Direct Play.	OS Headless (sin GUI), desactivación de suspensión por tapa (systemd logind).
Almacenamiento	Disco Interno + Expansión USB 3.0 Ext4	**I/O BOTTLENECK** HDDs mecánicos ralentizan la indexación masiva.	Montaje con flag *noatime*, base de datos en SSD/memoria rápida y backups Restic.
Display (Tab 4)	1.5 GB RAM, Android Legacy	**MEMORIA CRÍTICA** Navegadores modernos saturan la RAM.	LineageOS sin GApps, cliente web ligero en Vanilla JS con WebKit mínimo.
Batería Display	Li-ion conectada 24/7	**RIESGO TÉRMICO** Estrés químico e hinchamiento de celda.	Ciclo de carga acotado (20%-80%) mediante daemon Python y relé inteligente Wi-Fi.

3. Arquitectura del Sistema y Topología de Red

4. Stack de Software y Justificación Técnica

- **Sistema Operativo:** Ubuntu Server 24.04 LTS o Debian 12 Bookworm (consumo en reposo < 250 MB RAM).
- **Contenedores:** Docker Engine + Docker Compose (aislamiento modular de servicios y volúmenes persistentes).
- **Gestión Fotográfica (Immich):** Backend NestJS, PostgreSQL 16 con extensión *pgvector* para reconocimiento facial local y Redis para colas asíncronas.
- **Streaming Multimedia (Jellyfin):** 100% de código abierto, compatibilidad con códecs modernos y soporte para aceleración hardware (VA-API / Intel QuickSync).
- **Enrutamiento y TLS (Caddy):** Reverse proxy moderno con gestión automatizada de certificados y bajo uso de memoria.
- **Conectividad Remota (Tailscale):** Red mallada WireGuard para acceso seguro fuera de la red local sin exponer puertos al exterior.

5. Componentes de Desarrollo Propio (Valor para Portafolio/CV)

1. Microfrontend Kiosk para Hardware con Recursos Críticos

SPA ultraligera (< 5 MB RAM) desarrollada en Vanilla JavaScript y CSS nativo. Realiza llamadas a la API REST de Immich para consumir álbumes específicos con transiciones aceleradas por hardware y reciclaje de nodos en DOM para evitar fugas de memoria.

2. Subsistema de Telemetría y Carga Inteligente de Batería

Daemon en Python ejecutado en el servidor que extrae telemetría de la tablet vía ADB sobre Wi-Fi y orquesta el encendido/apagado de un enchufe inteligente (Shelly / Tasmota / Tuya) manteniendo la batería entre el 20% y el 80%.

6. Plan de Pruebas y Criterios de Aceptación

1. **Rendimiento en Carga Masiva:** Subida y transcodificación de miniaturas en segundo plano sin bloquear el hilo I/O del servidor.
2. **Streaming Direct Play:** Reproducción simultánea 1080p en Smart TV con uso medio de CPU < 15%.
3. **Estabilidad del Display:** Tiempo continuo de funcionamiento (MTBF) del marco digital > 30 días sin intervención manual.
4. **Resiliencia Eléctrica:** Recuperación automática de todos los contenedores y puntos de montaje ante reinicios intempestivos.